CSS GRID LAYOUT

Grade 11 • TechPro • Guided Activity Sheet

Name: _____ Section: _____ Date: _____

LEARNING TARGETS

- Explain the purpose of CSS Grid and identify grid containers, items, tracks, lines, cells, areas, and gaps.
- Create rows and columns using display: grid, grid-template-columns, grid-template-rows, fr, repeat(), and minmax().
- Control spacing and alignment of the grid and individual grid items.
- Place, span, name, and reorder grid items; build a responsive 12-column layout.
- Use @supports to apply Grid only when supported and provide a simple fallback.

TOOLS AND PREPARATION

- Use Visual Studio Code and a modern web browser.
- Create a folder named CSS_Grid_Activity. Inside it, create index.html and style.css.
- Link style.css to index.html and save both files in the same folder.

PART A — GRID BASICS AND CONTAINER

- Read the W3Schools CSS Grid and Grid Container lessons.
- In your notebook, define: grid container, grid item, row, column, track, grid line, cell, area, and gap.
- Set a wrapper element to display: grid. Add six child div elements and observe that each direct child becomes a grid item.
- Change display: grid to display: inline-grid. Describe how the container behaves differently.

PART B — TRACKS, GAPS, AND CONTAINER ALIGNMENT

- Set three equal columns with grid-template-columns: repeat(3, 1fr).
- Set two or more row tracks with grid-template-rows. Try fixed values such as 100px and flexible values such as 1fr.
- Experiment with auto, px, %, fr, repeat(), and minmax() to change track sizes.
- Set row-gap and column-gap separately. Then use gap as shorthand.
- Test justify-content and align-content to position the whole grid when extra space is available.
- Test justify-items and align-items to position items inside their own grid areas.

PART C — GRID ITEMS: PLACEMENT AND SPANNING

- Create six course cards: HTML, CSS, JavaScript, Java, Python, and Database.
- Make one card span two columns using grid-column: span 2.
- Make another card span two rows using grid-row: span 2.
- Use grid-column-start, grid-column-end, grid-row-start, or grid-row-end to place an item using grid-line numbers.
- Try placing an item by using the shorthand grid-column or grid-row.

PART D — NAMED GRID ITEMS AND AREAS

- Create a page layout with header, navigation, main content, sidebar, and footer.
- Use grid-template-areas to name the layout regions, for example: header, nav, main, side, and footer.
- Assign each page section with the matching grid-area name.
- Change the named-area arrangement and observe how the page layout changes without moving the HTML elements.

PART E — GRID ITEM ALIGNMENT AND ORDER

- Use justify-self and align-self to position one selected card inside its grid cell.
- Compare item-level alignment (justify-self / align-self) with container-level alignment (justify-items / align-items).
- Change the visual order of two cards using the order property.
- Before submission, restore a sensible reading order. Visual reordering should not create a confusing keyboard or screen-reader reading sequence.

PART F — RESPONSIVE 12-COLUMN GRID

- Create a 12-column layout using grid-template-columns: repeat(12, 1fr).
- Make the header, navigation, and footer span all 12 columns.
- On a wide screen, make the main content span 8 columns and the sidebar span 4 columns.
- At a screen width of 600px or less, make both main content and sidebar span all 12 columns.
- Test your page at desktop, tablet, and phone widths. Adjust gaps and card columns for readability.

PART G — @supports RULE

- Add @supports (display: grid) around the CSS Grid layout declarations.
- Before the @supports block, provide a basic fallback using normal block layout and padding.
- Test the page in your browser and explain what @supports checks and why a fallback can be useful.

STARTER FILES

- Type the following starter code. Then complete the CSS tasks in Parts A–G.

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>TechPro Learning Hub</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <div class="page-grid">
    <header class="header">TECHPRO LEARNING HUB</header>
    <nav class="navigation">Home | Courses | Projects | About</nav>
    <main class="main-content">
      <h1>Explore ICT Courses</h1>
      <div class="course-grid">
        <article class="course featured">HTML</article>
        <article class="course">CSS</article>
        <article class="course">JavaScript</article>
        <article class="course">Java</article>
        <article class="course">Python</article>
        <article class="course">Database</article>
      </div>
    </main>
    <aside class="sidebar">
      <h2>Announcements</h2>
      <p>Complete your CSS Grid activity and test your page on a phone-sized screen.</p>
    </aside>
    <footer class="footer">Quirino High School • TechPro Department</footer>
  </div>
</body>
</html>
```

style.css

```
/* Basic fallback: page remains readable without Grid */
.page-grid { display: block; }
```

```
.header, .navigation, .main-content, .sidebar, .footer {
  padding: 16px; margin-bottom: 10px;
}

/* Use Grid when supported */
@supports (display: grid) {
  .page-grid {
    display: grid;
    grid-template-columns: repeat(12, 1fr);
    gap: 12px;
  }
  .header, .navigation, .footer { grid-column: 1 / -1; }
  .main-content { grid-column: span 8; }
  .sidebar { grid-column: span 4; }

  .course-grid {
    display: grid;
    grid-template-columns: repeat(3, 1fr);
    gap: 12px;
  }
  .course {
    padding: 24px 12px;
    text-align: center;
    background: #f7eef1;
    border: 1px solid #7a1835;
  }
  .featured { grid-column: span 2; }
}

@media (max-width: 600px) {
  @supports (display: grid) {
    .main-content, .sidebar { grid-column: 1 / -1; }
    .course-grid { grid-template-columns: repeat(2, 1fr); }
  }
}
```

CHALLENGE PROJECT — TECHPRO LEARNING HUB

- Build a complete responsive ICT learning dashboard with a header, navigation, main course area, sidebar announcements, and footer.
- Include at least six course cards and make one card span more than one column.
- Use named grid areas for the main page layout.
- Demonstrate container alignment, individual item alignment, item order, and a 12-column grid.
- Use @supports with a basic fallback. Make sure the page remains readable on a narrow phone screen.
- Apply a consistent maroon, white, and light-gray color theme. Use readable font sizes and sufficient spacing.

SUBMISSION CHECKLIST

- index.html and style.css are saved in the CSS_Grid_Activity folder.
- The project demonstrates grid containers, tracks, gaps, and alignment.
- The project demonstrates item placement, spanning, naming, item alignment, and order.
- The page uses a responsive 12-column layout and has been tested at different screen widths.
- An @supports rule and a basic fallback are included.
- Submit the project folder and screenshots of the desktop and mobile layouts.

REFLECTION QUESTIONS

- How does CSS Grid arrange elements in two dimensions?

  _____

- What is the difference between a grid track and a grid line?

  _____

- What does the fr unit represent?
  _____
- How are gap, justify-content, and align-items different?
  _____
- When would you use grid-column or grid-row to span an item?
  _____
- What is the benefit of naming grid areas?
  _____
- Why should visual reordering be used carefully?
  _____
- What is the purpose of the @supports rule?
  _____

W3SCHOOLS REFERENCE LESSONS

- CSS Grid — https://www.w3schools.com/css/css_grid.asp
- Grid Container — https://www.w3schools.com/css/css_grid_container.asp
- Grid Tracks — https://www.w3schools.com/css/css_grid_tracks.asp
- Grid Gap — https://www.w3schools.com/css/css_grid_gap.asp
- Grid Alignment — https://www.w3schools.com/css/css_grid_align.asp
- Grid Items — https://www.w3schools.com/css/css_grid_item.asp
- Grid Item Names — https://www.w3schools.com/css/css_grid_item_name.asp
- Grid Item Alignment — https://www.w3schools.com/css/css_grid_item_align.asp
- Grid Item Order — https://www.w3schools.com/css/css_grid_item_order.asp
- 12-Column Grid — https://www.w3schools.com/css/css_grid_12column.asp
- @supports Rule — https://www.w3schools.com/css/css_supports_rule.asp